

pandas

A python library for performing relational operational on raw data files



pm jat @ daiict



Hope you know!

- `LIST`
  - Python `List` is a collection of values

- Dictionary
  - Map(K,V)

- Then, you have **`NumPy`**'s "**`NDArray`**"

NumPy's Array object - "NDArray"

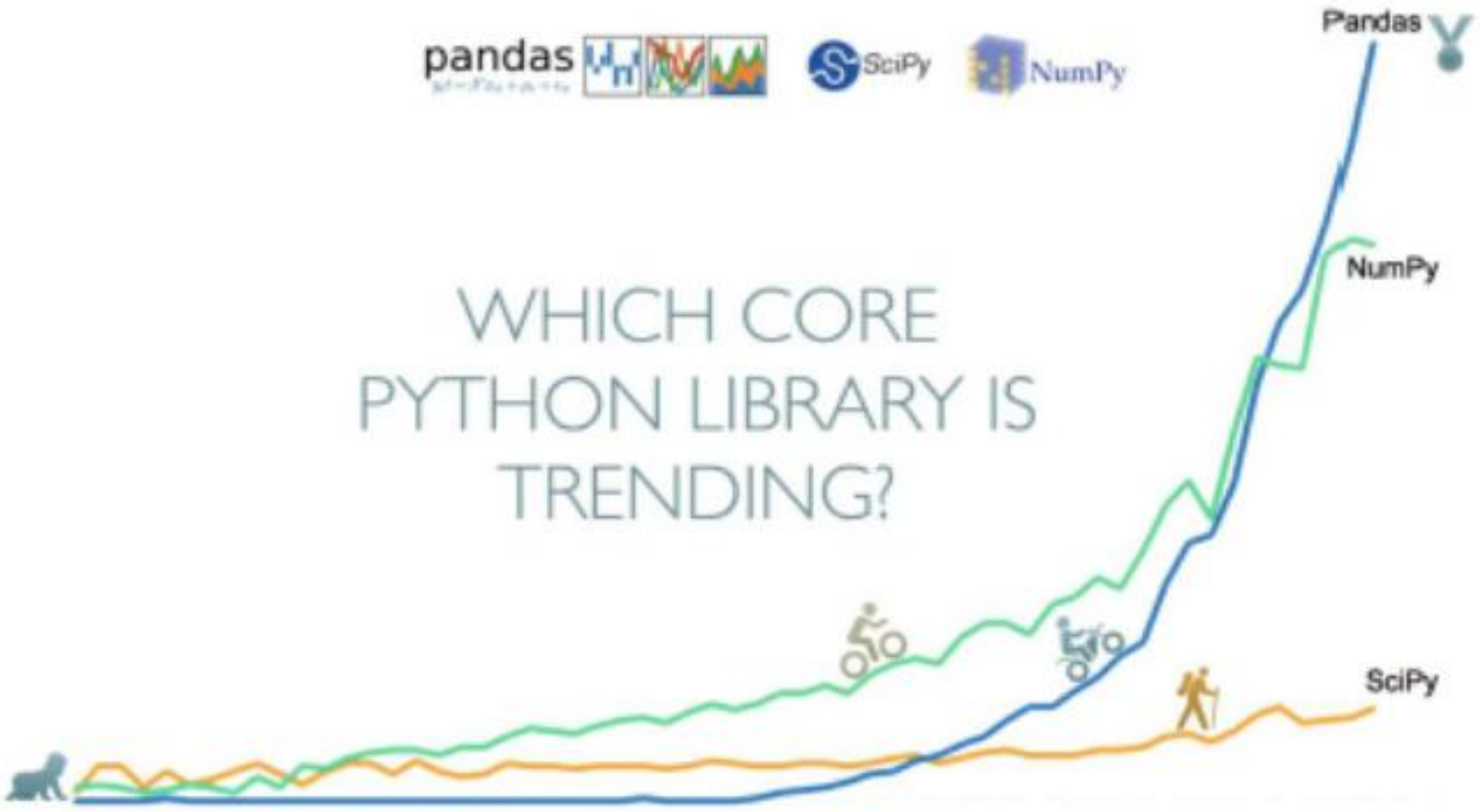
- Why "NumPy"?
  - **Efficient Storage**: internally stores data in a contiguous block of memory,
  - **Vectorized Processing**: operations perform complex computations on entire arrays without the need for Python for loops.
- The most import data object is "**`ndarray`**"
- "**N-D-array**", why?
  - fast array-oriented arithmetic operations.
  - Vectorized execution of mathematical functions on the array -> fast operations on entire arrays of data without having to write loops.
  - Tools for reading/writing array data to disk and working with memory-mapped files.
  - Efficient "Linear Algebra" operations

pandas – a Python data analysis library

- A snapshot from "https://pandas.pydata.org/"



pandas – a Python data analysis library



Source: Gupta, Pramod, and Anupam Bagchi. *Essentials of Python for Artificial Intelligence and Machine Learning*. Springer, 2024. (Chapters 5 and 6)



pandas – why so popular

- Provide Simple “Tabular data structure” like SQL tables
- Ability to perform “relational” and “analytical” operations through simple interface
  - SELECTION, PROJECTION, JOIN, UNION, Intersection, Difference
  - AGGREGATION
  - Statistical Summaries
  - Pivoting
  - Etc
- Input/Output integration with several data stores: CSV, JSON, SQL databases, etc.



pandas – data model

- Provides three main data structures
  - **`Series`** – a 1D array with named index access
  - **`Dataframe`** – a tabular view of data (a SQL table like structure)
  - **`Index`** – index structure

“Series” object

- **Pandas Series** is a
  - one-dimensional array like collection of “homogenous” data of a type
  - Indexed random access through “index” labels
- Has two parts (as shown in figure here) ==>
  - “index” – “label” list (ordered)
  - “data” – “vale” list (ordered) of some type
- Hands-on: “**Tutorial #1”** to understand more about it

Index	values
C002	Priya
C001	Rajesh
C003	Amit
C004	Deepak
C005	Meera
C006	Sunita
C007	Vikram
C008	Anil
C009	Sneha
C010	Neha
C011	Rakesh



Python “List” vs Panda “Series”

- Homogenous elements
- Labelled Index
- Vectorized Processing

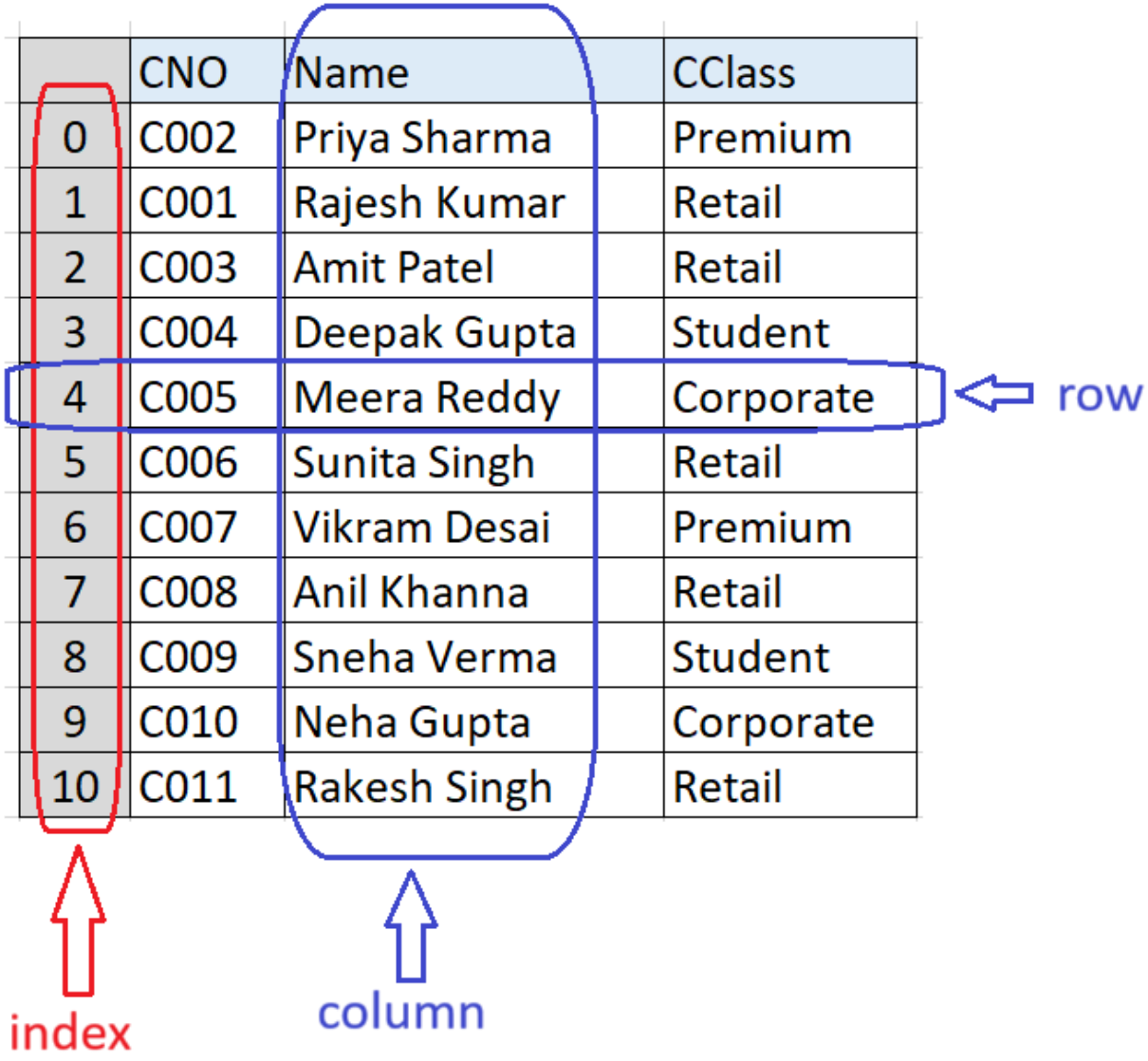


"Dataframe"

- The pandas "data-frame" attempts simulating a SQL table in python and allows performing all table operations
- Panda implements data-frame columns as a
  - "Collection (Dictionary) of a number of "Series" objects, and
  - have a "common index", called "columns"
- Elements in a column (like in a series) are of same type, however different columns can have different data types

“Dataframe” object

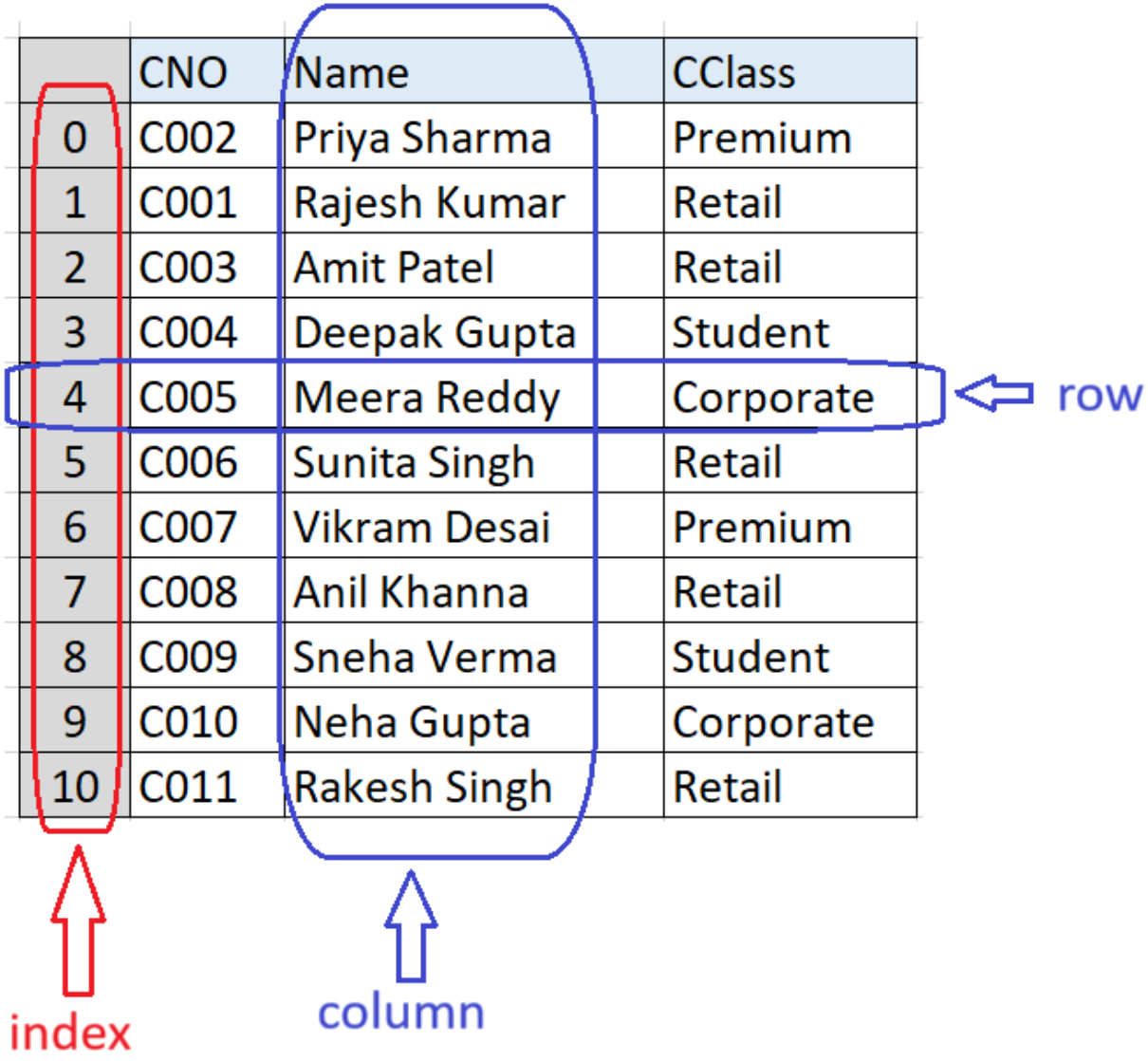
- Index
- Columns
  - A Collection of “Series” objects each representing a column
  - A shared “Index” – all columns share the same index (labels to have indexed access of column elements – values from all columns form a row
  - Any of the column can be made as index (typically the one that has unique-value)



“Dataframe” object

- Each Column would have
  - A list of values; can be accessed through “index-value” – referred as a label

- Hands-on:
  “**Tutorial #2” –Dataframe Basics**
  to understand more about dataframe objects



“Dataframe” operations

- SELECTION
  - Set of rows from the operand dataframe that meet the specified criteria
- PROJECTION
  - Select only specified columns

- Hands-on:
  “**Tutorial #3” – Dataframe Basic Operations**
  to understand more about these operations

AccNo	AccType	BID	Balance
ACC001	Current	B001	40050
ACC002	Current	B002	38350
ACC003	Current	B003	38500
ACC004	Current	B004	38100
ACC005	Current	B005	38050
ACC006	Current	B006	35550
ACC007	Current	B007	39550
ACC008	Current	B008	40450
ACC009	Current	B009	37050

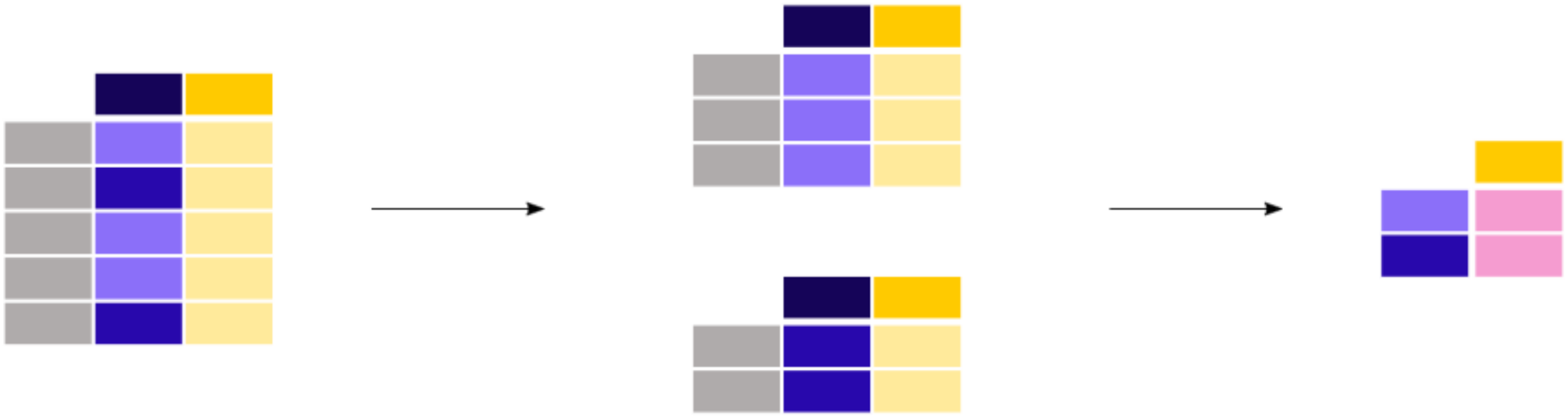
Summarization/aggregation operations

- Aggregations operations refers to perform a summarization operation on a “row-set”, typically for numeric columns, like balance here

AccNo	AccType	BID	Balance
ACC001	Current	B001	40050
ACC002	Current	B002	38350
ACC003	Current	B003	38500
ACC004	Current	B004	38100
ACC005	Current	B005	38050
ACC006	Current	B006	35550
ACC007	Current	B007	39550
ACC008	Current	B008	40450
ACC009	Current	B009	37050

“grouped” aggregation

- **Split** the data into groups
- **Apply** a function to each group independently
- **Combine** the results into a data structure
- Hands-on: “**Tutorial #4” – Aggregation Operations**



Summarization/aggregation operations

- Hands-on:
  “**Tutorial #4” – Summarization/aggregation operations**

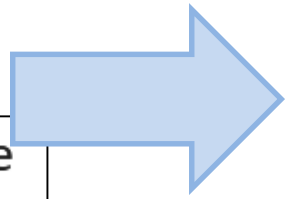
“split” --> “apply” (say mean) --> “combine”

acc_no	acc_type	bid	balance
ACC001	Current	B001	40050.0
ACC002	Current	B002	38350.0
ACC003	Current	B003	38500.0
.....			

acc_no	acc_type	bid	balance
ACC101	Fixed Deposit	B001	25000.0
ACC102	Fixed Deposit	B002	25000.0
ACC103	Fixed Deposit	B003	25000.0

	balance
	37600.0
	36500.0
	37850.0

acc_no	acc_type	bid	balance
ACC051	Savings	B001	44200.0
ACC052	Savings	B002	39950.0
ACC053	Savings	B003	37900.0
.....			

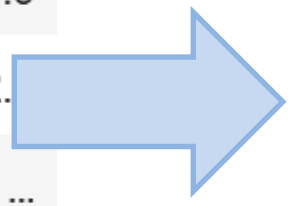


Current	40685.566406
Fixed Deposit	36000.000000
Salary	39815.261719
Savings	42503.183594

Pivoting

```
df2.pivot(index="bid", columns="acc_type", values="balance")
```

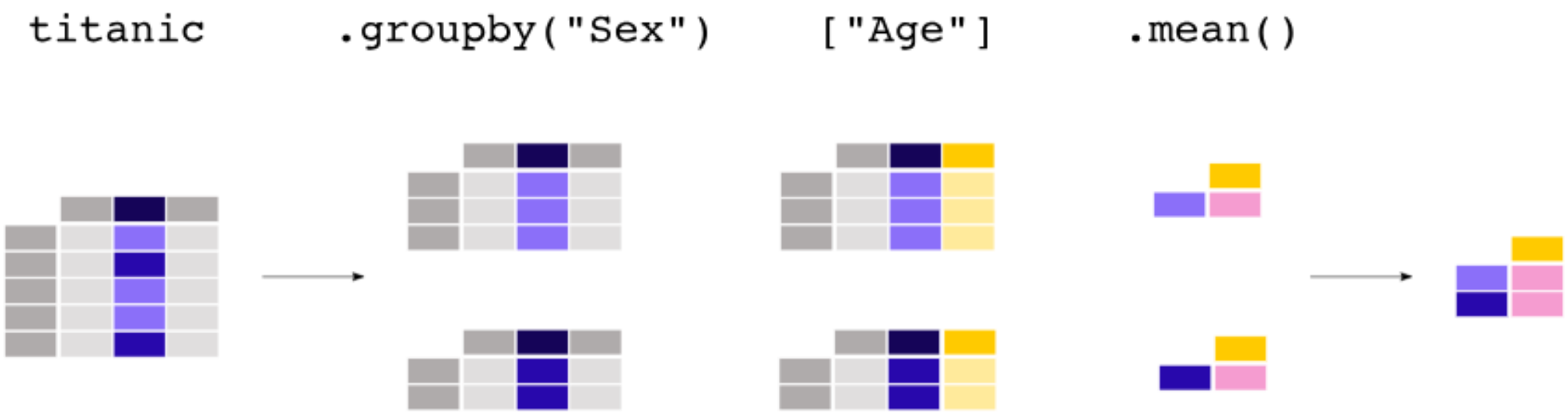
bid	acc_type	balance
B001	Current	34200.0
B001	Fixed Deposit	22500.0
B001	Salary	39600.0
B001	Savings	51260.0
B002	Current	41362.
...	...	...
B025	Savings	41810.0
B901	Current	37000.0
B901	Fixed Deposit	37000.0
B902	Current	42000.0
B902	Savings	35000.0



acc_type	Current	Fixed Deposit	Salary	Savings
bid				
B001	34200.000000	22500.000000	39600.000000	51260.000000
B002	41362.500000	39750.000000	41366.666667	39714.285714
B003	46100.000000	45300.000000	44125.000000	39525.000000
B004	39050.000000	37375.000000	35310.000000	43792.857143
B005	43300.000000	27750.000000	43862.500000	35875.000000
B006	35200.000000	37500.000000	39360.000000	32133.333333
B007	51440.000000	42750.000000	41266.666667	38237.500000
B008	38325.000000	25000.000000	39900.000000	44766.666667
B009	39840.000000	44000.000000	26416.666667	46940.000000

“grouped” aggregation:
“groupby” --> “projection” --> “aggregation”

```
titanic_df.groupby("Sex")["Age"].mean()
```





Pivoting

```
df3.pivot(index="bid", columns="acc_type", values=['sum', 'mean', 'median'])
```

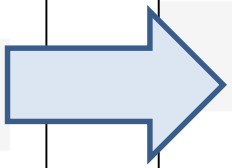
	sum				mean				
acc_type	Current	Fixed Deposit	Salary	Savings	Current	Fixed Deposit	Salary	Savings	Current
bid									
B001	136800.0	67500.0	118800.0	256300.0	34200.000000	22500.000000	39600.000000	51260.000000	38900.0
B002	165450.0	238500.0	248200.0	278000.0	41362.500000	39750.000000	41366.666667	39714.285714	37975.0
B003	138300.0	226500.0	176500.0	158100.0	46100.000000	45300.000000	44125.000000	39525.000000	38500.0
B004	195250.0	149500.0	176550.0	306550.0	39050.000000	37375.000000	35310.000000	43792.857143	38100.0
B005	129900.0	111000.0	175450.0	71750.0	43300.000000	27750.000000	43862.500000	35875.000000	38850.0
B006	70400.0	262500.0	196800.0	96400.0	35200.000000	37500.000000	39360.000000	32133.333333	35200.0



JOIN

acc_no	acc_type	bid	balance
ACC001	Current	B001	40050.0
ACC002	Current	B002	38350.0
ACC003	Current	B003	38500.0
ACC004	Current	B004	38100.0
ACC005	Current	B005	38050.0
ACC006	Current	B006	35550.0
ACC007	Current	B007	39550.0
ACC008	Current	B008	40450.0
ACC009	Current	B009	37050.0
ACC010	Current	B010	36100.0

accountno	cno
ACC001	C001
ACC001	C093
ACC001	C028
ACC002	C002
ACC003	C003
ACC004	C004
ACC005	C005
ACC005	C011
ACC005	C010
ACC006	C006



accountno	cno	acc_type	bid	balance
ACC001	C001	Current	B001	40050.0
ACC009	C001	Current	B009	37050.0
ACC142	C001	Fixed Deposit	B017	25000.0
ACC101	C001	Fixed Deposit	B001	25000.0
ACC269	C002	Salary	B005	27500.0
ACC268	C002	Savings	B025	32000.0
ACC002	C002	Current	B002	38350.0
ACC013	C002	Current	B013	37450.0
ACC349	C002	Fixed Deposit	B020	30000.0
ACC102	C002	Fixed Deposit	B002	25000.0



Summarization/aggregation operations

- Hands-on: **Tutorial #5**
  “JOIN and more on summarization/aggregation operations”

Resource - Pandas Documentation

- Home Page: http://pandas.pydata.org/docs/
- First Place to go: http://pandas.pydata.org/docs/getting_started/index.html
  - Introduces and provides a tour through Tutorials
- User Guide: https://pandas.pydata.org/docs/user_guide/index.html
- Tutorials: http://pandas.pydata.org/docs/getting_started/intro_tutorials/index.html
- API Reference: https://pandas.pydata.org/docs/reference/index.html
  - Provides a detailed description of fields and methods
- Development: https://pandas.pydata.org/docs/development/index.html
  - For professional developers and contributors
- Cheat Sheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf